PROFESSIONAL TRAINING CURRICULUM

THE COMPLETE
DATA ENGINEERING MASTERY ROADMAP

From beginner to industry-ready Data Engineer

12 phases • 9–12 months • one tool stack for every phase



Python · SQL · PostgreSQL · dbt · Airflow · Docker · AWS · Spark · Kafka · Terraform

Contents

Section	What you will find
Understand the role	What a Data Engineer actually builds, and the five layers of a data platform
The learning strategy	Four stages across twelve months, with realistic planning notes
Roadmap at a glance	All 12 phases with their core tools and flagship projects on one page
Your core tech stack	The small, consistent stack to master first
Phases 01–12	Learning modules, matching tools and platforms, a hands-on project, resources and an exit criterion for each phase
Your weekly study system	A 15-hour weekly pattern and a typical learning week
Readiness self-check	An eight-skill, 24-point self-assessment
Build a portfolio	Three portfolio pillars and what every repository should contain
Affordable learning strategy	Free-first tooling, what is worth paying for, and an installation checklist
Priorities and final rule	Tool-selection questions and the recommended learning order

Understand what you are training to become

Data Engineering is a field where knowing the tools is not enough. You can learn Python, SQL, Spark, Airflow, Kafka and AWS and still struggle to build a reliable data platform that a real business can use.

The difference between someone who knows data engineering tools and a competent Data Engineer is the ability to **design, build, operate, secure, troubleshoot and improve** systems that move raw data into trustworthy, usable information.

This roadmap works as a professional training curriculum: it starts from absolute fundamentals, introduces technical depth progressively and ends with production-style projects that demonstrate real engineering competence. It emphasizes Python, SQL, PostgreSQL, cloud infrastructure, orchestration, distributed processing, data modeling and production deployment, and it prioritizes affordable resources accessible to someone studying in Kenya.

A picture of the job

Imagine a supermarket chain with 50 stores across Kenya. Every day it generates POS sales transactions, stock movements, supplier deliveries and purchase orders, website interactions and financial records. Management wants to know which products sell fastest, which branches are losing money, when to restock and how to forecast demand.

The raw data is scattered across systems, formats and databases. Some records are duplicated, some arrive late, some contain errors. **Your job is to build the infrastructure that turns this scattered data into dependable information.**

The five layers of a data platform

1	**Data sources**	Applications, APIs, databases, CSV files, sensors and event streams.
2	**Ingestion and orchestration**	Extract and transport data; schedule and coordinate pipelines.
3	**Storage and transformation**	Store raw data, clean it, model it and make it consistent.
4	**Reliability and governance**	Validate data quality, control access, monitor failures and track lineage.
5	**Data consumers**	Power BI, analysts, business applications, data scientists and machine-learning systems.

> **The central idea**
>
> You are learning to engineer the whole system, not merely write scripts that copy files.

The learning strategy

The recommended path is a **12-month curriculum**, with the option to reach entry-level readiness in 6–9 months if you already have programming fundamentals and can study consistently.

MONTHS
01–03
Foundation

Python, SQL, Git, Linux, networking and relational databases.

MONTHS
04–06
Core engineering

ETL/ELT, dimensional modeling, dbt, Airflow, Docker and automated testing.

MONTHS
07–09
Cloud and scale

Object storage, cloud data warehouses, Spark, streaming fundamentals and infrastructure as code.

MONTHS
10–12
Professional readiness

Production capstone, observability, security, performance, documentation and interview preparation.

These are planning estimates, not guarantees. Your progress depends on your existing knowledge, available study hours, project complexity and how independently you can solve problems.

Roadmap at a glance

Every phase pairs a learning goal with the tools that implement it and a project that proves it.

#	Month	Phase focus	Core tools and platforms	Flagship project
01	1	Foundations of computing and data	VS Code, Ubuntu on WSL2, Git, GitHub	Documented repository
02	2	Python for data engineering	Python, pandas, PyArrow, pytest, Ruff	Public API to structured dataset
03	3	SQL and database engineering	PostgreSQL, DBeaver, psql	Retail database and star schema
04	4	ETL/ELT and pipeline reliability	Python, SQLAlchemy, Polars, Pandera	Robust ETL/ELT pipeline
05	5	Warehousing and transformation	dbt Core, PostgreSQL, Power BI / Metabase	Analytical warehouse
06	6	Orchestration and Docker	Airflow, Docker, Docker Compose	Scheduled end-to-end pipeline
07	7	Cloud data engineering	AWS (S3, IAM, Glue, Redshift), Terraform / OpenTofu	Cloud analytics pipeline
08	8	Distributed processing	PySpark, DuckDB, Parquet, Iceberg / Delta	Distributed sales processing
09	9	Streaming and event-driven pipelines	Kafka, Spark Structured Streaming	Real-time event pipeline
10	10	Quality, governance, observability	Great Expectations, Pandera, Prometheus, Grafana	Quality-control layer

#	Month	Phase focus	Core tools and platforms	Flagship project
11	11	Production, CI/CD, system design	GitHub Actions, Trivy, pip-audit, Docker	Productionized pipeline
12	12	Industry-grade capstone	The full stack, used where justified	Kenya Retail Intelligence Data Platform

Your core tech stack

Build a small, consistent stack first, then introduce more advanced tools as your projects demand them. Most foundations can be learned with open-source software on your laptop; you do not need a boot camp, multiple cloud subscriptions or five competing tools for the same task.

Python Extraction, automation, data cleaning, APIs and pipeline logic. **Free and open source**	**PostgreSQL + DBeaver** Relational storage, SQL, transactions, database design and analytics queries. **Free local setup**	**Git + GitHub** Version control, project documentation, collaboration and CI/CD. **Free tier available**
Docker + Compose Reproducible environments for databases, Airflow, Kafka and other services. **Free core tooling**	**Airflow + dbt** Workflow orchestration and SQL transformation engineering. **Open-source core**	**AWS** Cloud storage, compute, identity, monitoring and managed data services. **Some services are paid**

This stack is enough to begin serious projects. Spark, Kafka, Terraform and other specialized technologies come later, when your foundations are strong enough to understand why they exist.

The phase-by-phase roadmap

Each phase below follows the same layout so you can navigate quickly:

- **What you will learn:** the modules and concepts, in the order to study them
- **Tools, platforms and tech stack:** the recommended tool for each task, with a free alternative or cost note
- **Hands-on project:** the build that proves the phase
- **Resources and exit criterion:** where to learn, and how to know you are ready to move on

How to read the tool tables

Tools are listed with their free or lower-cost options so you can begin locally and spend money only when a project genuinely requires it. Pricing and free-tier terms change, so verify current terms before committing.

PHASE 01

The foundations of computing and data

Month 1 | Estimated effort: 35–45 hours | Foundation

Understand the problems data tools solve, then build the command-line, networking and version-control fluency every later phase depends on.

What you will learn

Module 1.1 — Understand the data ecosystem

Before learning tools, understand the problems they solve.

- What is data?
- Structured, semi-structured and unstructured data
- Operational databases versus analytical systems: **OLTP vs OLAP**
- Data warehouses, data lakes and lakehouses
- Batch versus stream processing; **ETL vs ELT**
- Data pipelines and data workflows
- The roles of Data Analyst, Analytics Engineer, Data Engineer, Data Architect and Data Scientist

Example: a bank's transaction system is optimised to record individual transfers quickly. Its analytical platform answers questions such as how transaction volumes changed across counties over five years. Different purposes, related data.

Module 1.2 — Linux and the command line

Learn Ubuntu Linux, preferably through WSL2 on Windows or a Linux virtual machine.

- Files, directories, paths, permissions and ownership
- Core commands: `pwd`, `ls`, `cd`, `mkdir`, `cp`, `mv`, `rm`, `cat`, `less`, `grep`, `find`, `head`, `tail`, `sort`, `cut`, `wc`
- Pipes and redirection; environment variables and shell configuration
- Processes, ports, services, logs and exit codes
- Bash scripting and scheduling
- SSH, public/private keys and remote servers
- Package management and basic troubleshooting

Exercise: download a retail-sales CSV, inspect it from the terminal, find missing values or malformed rows, count records and redirect the results to a report file.

Module 1.3 — Networking fundamentals

- IP addresses, subnet masks and DNS
- TCP versus UDP; ports and sockets
- HTTP, HTTPS, REST APIs and JSON
- TLS certificates, firewalls and basic routing
- Client-server architecture

Why it matters: a pipeline that cannot connect to a database may have a credential, DNS, firewall or port problem. A competent engineer investigates the actual cause rather than randomly changing code.

Module 1.4 — Git and GitHub

- Repositories, commits, branches, merges and pull requests
- `.gitignore`, meaningful commit messages and README documentation
- GitHub Issues and project tracking
- Merge-conflict resolution
- Secret management: never commit credentials

Tools, platforms and tech stack

Task	Recommended tool	Free alternative / cost
Code editor	Visual Studio Code	Free
Linux environment	Ubuntu on WSL2	Free
Terminal and shell	Windows Terminal + Bash	Free
Version control	Git	Free
Remote repositories	GitHub	Free plan
API inspection	Postman	Free plan; `curl` is a free alternative
Network troubleshooting	`ping`, `nslookup`, `curl`, `ss`	Free system utilities
Diagrams	diagrams.net	Free

Recommendation: use VS Code, Ubuntu on WSL2, Git and GitHub from the very beginning. This combination supports most of your later projects.

Hands-on project

Milestone — your first documented repository

- Create a GitHub repository for your Data Engineering journey containing Linux exercises, sample data, Python scripts and SQL queries
- Install Ubuntu through WSL2 and document your environment setup
- Draw the architecture of a simple data pipeline

Recommended resources

The Missing Semester | GitHub Skills | Ubuntu documentation | VS Code | WSL | Git | diagrams.net

Exit criterion

You can work comfortably in a Linux terminal, diagnose a connection failure by layer, and publish a documented Git repository without committing secrets.

PHASE 02

Python for professional data engineering

Month 2 | Estimated effort: 45–55 hours | Foundation

Python is your primary automation and pipeline language. The aim is not syntax; it is software that stays understandable when datasets grow, inputs break or systems fail.

What you will learn

Module 2.1 — Python fundamentals

1. Variables, types, operators, strings and input/output
2. Conditions, loops and comprehensions
3. Lists, tuples, sets and dictionaries
4. Functions, parameters, return values and scope
5. Exceptions and error handling
6. Reading and writing CSV, JSON and text files
7. Modules, packages, imports and virtual environments
8. Type hints, docstrings and debugging

Module 2.2 — Intermediate Python

- Iterators, generators, context managers, decorators and dataclasses
- Object-oriented programming where useful
- Logging, configuration and dependency management
- Unit testing with `pytest`
- Dates, time zones and timestamps
- API requests, pagination, rate limits and retries

Do not rush into advanced Python features before you can write and test ordinary functions reliably.

Module 2.3 — Data processing

- CSV, JSON, JSON Lines and Parquet
- `pandas` for exploratory and moderate-sized tabular transformations
- `pyarrow` and the basics of columnar storage
- Data types, null values, duplicates and malformed inputs
- Streaming file processing instead of loading everything into memory
- Memory and runtime profiling; incremental processing and idempotent scripts

A practical example

An API returns 100,000 sales records. Your pipeline must:

1. Request the data from the API
2. Check the response status and structure
3. Validate required fields
4. Normalize timestamps and numeric values

5. Handle missing or malformed records
6. Write the results to a durable destination
7. Record how many rows succeeded or failed
8. Log enough information to diagnose problems later

A script that completes the first five steps but silently loses records is not a reliable pipeline.

Tools, platforms and tech stack

Task	Recommended tool	Free alternative / cost
Python runtime	Python 3	Free
Python environments	venv + pip	Free
Alternative environment manager	Miniforge	Free and open source
Data manipulation	pandas	Free
Columnar data processing	PyArrow	Free
Unit testing	pytest	Free
Linting and formatting	Ruff	Free
API requests	Requests	Free
API exploration	Postman	Free plan or curl
Interactive notebooks	JupyterLab	Free

Paid option to avoid: you do not need a paid Python IDE, notebook platform or data-cleaning subscription. Local Python and Jupyter are sufficient.

Hands-on project

Project 1 — Public API to structured dataset

Use a public weather, economic, transport or other openly accessible API. Deliverables:

- A reusable API client with pagination and retry handling
- Raw JSON saved with an extraction timestamp
- A cleaned CSV or Parquet dataset
- Logging and a summary of rejected records
- Automated tests for transformation logic
- A GitHub README with setup instructions and sample output

Completion test: if the API is temporarily unavailable, your script fails clearly, preserves previously collected data and provides enough logs to diagnose the issue.

Recommended resources

Python official tutorial | Automate the Boring Stuff with Python | pandas | PyArrow | pytest | Jupyter

Exit criterion

You can build a Python data extraction program from scratch, test it, and explain its failure-handling strategy.

PHASE 03

SQL and database engineering

Month 3 | Estimated effort: 50–60 hours | Foundation

If Python is your automation language, SQL is one of your most important tools. Weak SQL undermines transformations, debugging, analytics models and performance work.

What you will learn

Module 3.1 — Relational database fundamentals

- Tables, rows, columns, schemas and data types
- Primary keys, foreign keys and relationships (one-to-one, one-to-many, many-to-many)
- Constraints and data integrity
- Normalization: 1NF, 2NF, 3NF
- Transactions and ACID properties
- Indexes and uniqueness; users, roles and privileges

Understand why databases enforce rules: if two orders accidentally share an identifier that should be unique, downstream revenue calculations may become incorrect.

Module 3.2 — SQL from beginner to advanced

Level	Topics
1 — Querying	`SELECT`, `WHERE`, `ORDER BY`, `LIMIT`, `DISTINCT`, `CASE`, `NULL` handling
2 — Combining data	Aggregations, `GROUP BY`, `HAVING`, joins, set operations, subqueries and CTEs
3 — Analytical SQL	Window functions, ranking, running totals, deduplication, date arithmetic and sessionization
4 — Performance	Indexes, execution plans, query selectivity, join strategies, partitioning and query tuning

Module 3.3 — Learn PostgreSQL deeply

- Database creation and schema management; importing CSV data
- Views and materialized views; index design
- Transactions and isolation; deadlocks, locks and concurrent operations
- `EXPLAIN` and `EXPLAIN ANALYZE`; query planning
- Backup and restore; roles and access control
- Connection pooling fundamentals; Change Data Capture (CDC) concepts

Use DBeaver as a graphical client if you prefer it, but learn the command line through `psql` too.

Module 3.4 — Data modeling

- Fact tables and dimension tables; star and snowflake schemas
- Grain: exactly what one row represents
- Surrogate and natural keys
- Slowly Changing Dimensions (SCD Types 1 and 2)
- Late-arriving facts and dimensions; snapshot tables

- Incremental loads and historical tracking

Critical principle

Define the grain before designing the fact table. Is one row one transaction, one order line, one product per day, or one inventory snapshot per branch? Getting this wrong can double-count revenue or inventory.

Tools, platforms and tech stack

Task	Recommended tool	Free alternative / cost
Relational database	PostgreSQL	Free and open source
Database GUI	DBeaver Community	Free and open source
SQL command line	`psql`	Included with PostgreSQL
ER diagrams	diagrams.net	Free
Sample datasets	Kaggle	Free account for many datasets
SQL exercises	SQLBolt	Free

Should you use a cloud database already?

Not necessarily. Begin with local PostgreSQL. To practise remote connections later, evaluate managed PostgreSQL services such as Neon or Supabase, checking their current free-plan limits. A remote database teaches networking and cloud connectivity, but it should not replace your understanding of database administration.

Hands-on project

Project 2 — Build a retail database

Build a PostgreSQL database for a Kenyan retail business with `customers`, `products`, `stores`, `orders`, `order_items`, `payments` and `inventory_movements`. Then design an analytical star schema with a sales fact table and appropriate dimensions. Answer questions such as:

- Which branches generate the most revenue, and what is the monthly growth rate?
- Which products have high sales but frequent stockouts?
- Which customers have not purchased recently, and which products show declining sales?
- Does reported revenue reconcile with the source transactions?

Include indexes, constraints, SQL tests, an entity-relationship diagram and query-performance evidence.

Recommended resources

PostgreSQL documentation | SQLBolt | Use The Index, Luke | DBeaver | Kaggle datasets

Exit criterion

You can design a relational database, write complex analytical SQL, explain an execution plan, and identify duplicate-counting errors.

PHASE 04

Data ingestion, ETL/ELT and pipeline reliability

Month 4 | Estimated effort: 40–50 hours | Core engineering

This is where your foundation becomes actual Data Engineering: moving data reliably from source to destination.

What you will learn

Module 4.1 — Understand pipeline design

Stage	Meaning
Extraction	Obtaining data from a source
Ingestion	Bringing data into a platform
Transformation	Cleaning, joining, enriching and restructuring data
Loading	Writing data into a destination
Orchestration	Coordinating tasks, dependencies, schedules and retries

Also learn:

- Full versus incremental extraction; batch loading
- Watermarks and high-water marks; CDC fundamentals
- API pagination and schema evolution
- Backfills and reprocessing
- Data freshness and latency
- Exactly-once claims versus practical delivery guarantees

Module 4.2 — Build reliable pipelines

Pattern	Principle
Idempotency	Running a pipeline twice must not create duplicate business records or corrupt the result.
Retries and backoff	Retry temporary failures carefully, without overwhelming the source system.
Checkpointing	A long-running process can resume from a known point where possible.
Dead-letter handling	Isolate and investigate bad records instead of letting them disappear silently.
Reconciliation	Compare source and destination counts, amounts, identifiers or checksums.
Atomicity	Never expose a half-written dataset as if it were complete.
Schema validation	Detect unexpected types, missing required columns and incompatible source changes.

Module 4.3 — Understand data formats

Format	Typical strengths
CSV	Simple interchange and human readability
JSON	APIs and nested semi-structured data
JSONL	Record-by-record processing and logs
Parquet	Columnar analytical storage and compression
Avro	Schema-aware record serialization and streaming ecosystems

Understand row-oriented versus columnar storage, compression, schema metadata, partitioning and the cost of reading unnecessary columns.

Tools, platforms and tech stack

Task	Recommended tool	Free alternative / cost
Extraction and transformation	Python	Free
API ingestion	Requests or HTTPX	Free
Database connectivity	SQLAlchemy + Psycopg	Free
Tabular transformation	pandas	Free
Larger local datasets	Polars	Free and open source
Data serialization	PyArrow	Free
Validation	Pandera or custom Python checks	Free
Tests	pytest	Free
Logging	Python logging	Built in
Workflow configuration	YAML / JSON	Free

You do not need a paid integration platform to learn ETL. Python, PostgreSQL and good testing practices teach the underlying engineering concepts.

Hands-on project

Project 3 — Build a robust ETL/ELT pipeline

Create a daily pipeline that extracts source data, stores an immutable raw copy, transforms it and loads it into PostgreSQL. It should:

1. Extract from an API or source database
2. Preserve raw records for recovery
3. Validate schemas and required fields
4. Deduplicate records using a defined business key
5. Load data incrementally
6. Log row counts and execution times
7. Recover from partial failures

8. Produce a clean analytical dataset
9. Re-run safely for the same date range

Completion test: simulate a failed run, a duplicated batch, a malformed record and a late-arriving update. Demonstrate recovery without silently losing data or double-counting results.

Recommended resources

SQLAlchemy | Psycopg | Polars | Pandera

Exit criterion

You can build an incremental, idempotent pipeline that survives failures, duplicates and bad records, and prove it with reconciliation.

PHASE

05

Data warehousing and transformation engineering

Month 5 | Estimated effort: 40–50 hours | Core engineering

A warehouse is not merely a large database. It is a curated environment where data is organised so analytical questions are answered consistently and efficiently.

What you will learn

Module 5.1 — Warehouse architecture

- Operational databases versus analytical warehouses
- Staging, intermediate and presentation layers; raw, validated and curated data
- Fact and dimension modeling; data marts
- Partitioning and clustering
- Slowly Changing Dimensions, snapshotting and historical data
- Metrics and business definitions
- Data contracts between producers and consumers

A lake, a warehouse and a lakehouse are architectural approaches, not interchangeable names for one thing.

Module 5.2 — Learn dbt

Use dbt to develop maintainable SQL transformation projects.

- Models and model dependencies; sources and staging models
- Refactoring repeated SQL
- Tests for uniqueness, non-null values, accepted values and relationships
- Documentation and lineage
- Incremental models and snapshots
- Jinja and macros
- Environments, deployment workflows and source freshness checks

A useful pattern: keep extraction code responsible for moving data and use dbt for warehouse transformations. They solve different problems and complement each other.

Tools, platforms and tech stack

Task	Recommended tool	Free alternative / cost
Warehouse for learning	PostgreSQL	Free
SQL transformations	dbt Core	Free and open source
Model tests	dbt tests	Included in the open-source workflow
Data documentation	dbt documentation	Free
Data lineage	dbt dependency graph	Free
Data modeling diagrams	diagrams.net	Free

Task	Recommended tool	Free alternative / cost
BI dashboard	Power BI Desktop	Free desktop authoring; sharing options may require payment

Important distinction

Open-source dbt is sufficient to learn transformation engineering; some managed dbt platform features are commercial. Start with local execution and PostgreSQL. Power BI Desktop is free to author in, but publishing and sharing through Microsoft's hosted service can require a paid licence. Alternatives: Metabase Open Source and Apache Superset.

Hands-on project

Project 4 — Create an analytical warehouse

Use the retail database from Phase 3 and build:

- A staging layer that standardizes source records, and intermediate models for business logic
- A sales fact table with product, customer, store and date dimensions
- Incremental transformation models and data-quality tests
- A documented lineage graph
- A small Power BI (or Metabase) dashboard using the curated models

Include business rules for cancellations, refunds, discounts, taxes and duplicate transactions. Explain what your revenue metric actually measures.

Recommended resources

dbt Learn | dbt documentation | Power BI Desktop

Exit criterion

You can create a tested, documented warehouse transformation project whose business metrics are consistent and explainable.

PHASE 06

Workflow orchestration and Docker

Month 6 | Estimated effort: 45–55 hours | Core engineering

A script may work on your laptop. A business pipeline must also run on schedule, recover from failures, respect dependencies and tell someone when something breaks.

What you will learn

Module 6.1 — Workflow orchestration with Airflow

Learn Apache Airflow after you can already build ordinary Python pipelines.

- DAGs, task dependencies, operators, tasks and hooks
- Scheduling and logical dates
- Retries, timeouts and failure callbacks; branching
- Backfills and catchup behavior
- Sensors and deferrable tasks
- Connections and secrets; task logs and execution history
- XCom, and when not to use it for large data
- Dynamic task mapping and data-aware scheduling concepts
- Testing DAG definitions and task logic

> **Important**
>
> Airflow coordinates work. It is not itself a database, a distributed processing engine or a substitute for reliable transformation code.

Module 6.2 — Docker and containerization

- Images versus containers; Dockerfiles, layers and image builds
- Volumes and persistent data
- Container networking and environment variables
- Compose files, health checks, logs and resource limits
- Container registries
- Non-root execution and secret handling

Understand why data can disappear when it is written only to an ephemeral container filesystem.

Tools, platforms and tech stack

Task	Recommended tool	Free alternative / cost
Orchestration	Apache Airflow	Free and open source
Container runtime	Docker Engine	Free core engine
Container GUI on Windows	Docker Desktop	Free for personal use, education and qualifying uses; some commercial organizations need paid plans

Task	Recommended tool	Free alternative / cost
Multi-container environments	Docker Compose	Free
Database service	PostgreSQL container	Free
SQL transformation	dbt Core	Free
Notifications	Email or webhook integration	Often free to start
Workflow visualization	Airflow UI	Included with Airflow

What if Docker is too demanding for your laptop?

Airflow's full environment can consume considerable RAM and CPU. Work progressively:

1. Develop and test Python tasks independently
2. Run PostgreSQL locally
3. Use Docker Compose when ready to orchestrate multiple services
4. Keep datasets small
5. Stop containers when not in use

For a lighter first step, learn Linux `cron`. It is useful, but it does not replace Airflow's dependency management, retries, backfills and observability.

Hands-on project

Project 5 — Schedule an end-to-end pipeline

Use Airflow, running locally through Docker Compose, to coordinate extraction, raw-data storage, loading into PostgreSQL, dbt transformations, data-quality checks and a final success/failure notification. Document the architecture and resource requirements, then demonstrate:

- A successful scheduled run
- A failed task and retry
- A safe rerun and a historical backfill
- A task dependency failure
- Logs that help identify the root cause

Recommended resources

Apache Airflow | Docker Get Started | Docker Compose

Exit criterion

You can run and troubleshoot a multi-step data workflow without manually executing each script.

PHASE 07

Cloud data engineering

Month 7 | Estimated effort: 45–55 hours | Cloud and scale

Cloud computing is where you learn to operate data systems beyond your laptop. Learn the concepts on one platform, then transfer them to the others.

What you will learn

Module 7.1 — Cloud fundamentals

- Regions and availability zones
- Compute, storage and networking; virtual machines and managed services
- Object storage versus block and file storage
- Identity and access management
- Public and private networking
- Encryption, key management, secrets and credential rotation
- Billing, quotas and resource limits
- Availability, durability and disaster recovery

Module 7.2 — Recommended cloud platform: AWS

Given an interest in Data Engineering and Cloud Analytics, learn AWS first while keeping the option to specialize in Azure later.

Service	Role	What to learn
Amazon S3	Object storage	Buckets, objects, IAM policies, encryption, lifecycle rules, versioning, partitioned datasets and Parquet files
AWS Glue	Data integration	Managed ETL, metadata catalogs, schema discovery, and when a managed service beats self-managed code
Amazon Redshift	Analytical warehousing	Warehouse workloads, distribution and sort strategies, query performance, workload management basics
AWS IAM	Access control	Users, roles, policies, least privilege and temporary credentials
Supporting services	Compute and operations	EC2, VPC, CloudWatch, Secrets Manager, Lambda and managed workflow services

Do not start by learning every AWS service. Understand how a few services work together to deliver a complete data platform.

Module 7.3 — Infrastructure as Code

- Terraform providers and resources; variables and outputs
- State and state protection
- Modules, plans and applies; resource dependencies
- Secrets and sensitive values
- Environment separation; safe changes and resource destruction

You should be able to provision a small cloud environment from version-controlled configuration rather than clicking through every setting.

Tools, platforms and tech stack — Recommended AWS stack

Requirement	Primary AWS tool	Free or lower-cost alternative
Object storage	Amazon S3	MinIO locally; LocalStack for selected local AWS service emulation
Relational database	Amazon RDS for PostgreSQL	Local PostgreSQL; a free-tier managed database if currently eligible
Data warehouse	Amazon Redshift	PostgreSQL for foundational learning; a trial or limited-free warehouse where available
Managed ETL	AWS Glue	Python, PySpark and dbt
Compute	Amazon EC2 / Lambda	Local Linux, Docker and Python
Permissions	AWS IAM	Practise concepts locally; real cloud policies need a cloud account
Logs and metrics	Amazon CloudWatch	Python logging, Prometheus and Grafana
Secrets	AWS Secrets Manager	Environment variables for local development
Infrastructure as code	Terraform	OpenTofu, a free and open-source alternative

AWS free access

As of October 2026, AWS advertises a Free account plan with an initial $100 credit, opportunities to earn more, and access to selected services for up to six months. The Free plan restricts certain services and a paid account has different billing implications. Check current terms at aws.amazon.com/free before choosing services.

Recommended cloud learning sequence

1. Create an S3-style data lake locally using MinIO and Parquet files
2. Learn AWS IAM, S3 and basic networking concepts
3. Deploy a small ingestion workload and store its output in S3
4. Learn a managed database or analytical warehouse only when needed
5. Use Terraform or OpenTofu to provision reproducible infrastructure
6. Measure costs, verify permissions and destroy resources when finished

Should you choose Azure instead?

Azure is a reasonable alternative, especially for Microsoft-oriented data roles. Its equivalent learning stack:

AWS	Azure equivalent
Amazon S3	Azure Data Lake Storage Gen2
AWS Glue	Azure Data Factory / Azure Databricks
Amazon Redshift	Microsoft Fabric Warehouse / Azure Synapse Analytics, depending on architecture

AWS	Azure equivalent
AWS IAM	Microsoft Entra ID and Azure RBAC
CloudWatch	Azure Monitor
Terraform	Terraform or OpenTofu

Azure's free-account offer advertises $200 credit for the first 30 days for eligible new customers plus specified free service quantities afterward. Not every service is free. **Recommendation:** pick AWS as your primary cloud; learn Azure equivalents later rather than splitting attention across two clouds.

Hands-on project

Project 6 — Cloud-based analytics pipeline

Build a platform that:

1. Ingests data from a public API
2. Saves raw records in S3
3. Organizes clean data in Parquet format
4. Uses an appropriate transformation process
5. Loads or exposes curated data for analytical queries
6. Enforces access permissions
7. Tracks failures and processing costs
8. Includes infrastructure configuration and teardown instructions

Start with a local version and deploy only the components you need. A project does not need an expensive managed cluster to demonstrate sound architecture.

Recommended resources

AWS Skill Builder | AWS Free Tier | Terraform tutorials | OpenTofu | Azure Free Account

Cloud offers and service charges change. Verify current pricing, set budget alerts and destroy resources after practice. A free account does not mean every service or usage level is free.

Exit criterion

You can provision a small, secure, cost-aware cloud data environment from code, run a pipeline on it, and tear it down cleanly.

PHASE 08

Distributed data processing with Apache Spark

Month 8 | Estimated effort: 40–50 hours | Cloud and scale

A common mistake is learning Spark before understanding SQL, data formats and ordinary data processing. First understand why a dataset becomes hard to process on one machine.

What you will learn

Module 8.1 — Distributed systems fundamentals

Imagine processing 500 GB of sales history. A single laptop may lack the memory or compute to handle it efficiently. Distributed processing divides work among executors, at the cost of coordination and infrastructure complexity.

- Horizontal versus vertical scaling; parallelism and concurrency
- Partitioning and shuffling; network transfer and data locality
- Replication and fault tolerance; distributed task execution
- Skewed data and uneven workloads
- Eventual consistency where applicable
- The trade-offs of distributed processing

Module 8.2 — Apache Spark

Master these areas in sequence:

1. Architecture: driver, executors, jobs, stages and tasks
2. DataFrames and Spark SQL
3. Reading and writing Parquet, CSV and JSON
4. Transformations versus actions; lazy evaluation
5. Narrow versus wide transformations
6. Partitioning and shuffle behavior
7. Joins and broadcast joins
8. Caching and persistence
9. Execution plans and the Spark UI
10. Adaptive Query Execution
11. Data skew, file sizing and small-file problems
12. Cluster configuration and resource management
13. Structured Streaming fundamentals

Learn PySpark alongside Spark SQL: use SQL for declarative transformations and Python where it improves clarity or flexibility.

Module 8.3 — Lakehouse concepts

Understand the problems solved by open table formats: **Apache Iceberg**, **Delta Lake** and **Apache Hudi**. Study table metadata, schema evolution, snapshots, concurrent writes and incremental reads. You need not implement all three; understand their shared goals and key differences.

Tools, platforms and tech stack

Task	Recommended tool	Free alternative / cost
Distributed processing	Apache Spark	Free and open source
Python interface	PySpark	Free
SQL processing	Spark SQL	Included
Local analytical engine	DuckDB	Free and open source
Columnar files	Parquet + PyArrow	Free
Local object storage	MinIO	Free to self-host; review licensing and current terms
Table formats	Apache Iceberg or Delta Lake	Open-source projects; hosting may cost money
Performance inspection	Spark UI	Included with Spark
Development notebooks	JupyterLab	Free

Spark or DuckDB first?

Use DuckDB for fast local analytical SQL over CSV and Parquet. Learn Spark when you need distributed execution, cluster-oriented processing or its broader ecosystem. Spark runs locally, so you do not need to rent a cluster to learn it, and you should avoid paying for managed Spark clusters while learning the fundamentals.

Hands-on project

Project 7 — Distributed sales processing

Use a public retail dataset and build a Spark job that:

- Processes historical transactions and handles missing or invalid records
- Deduplicates data using an explicit rule
- Joins multiple datasets
- Writes partitioned Parquet output
- Computes daily and monthly sales aggregates
- Produces data-quality metrics
- Includes execution plans and performance observations

Compare output and runtime against a reasonable single-machine implementation. Explain when Spark adds value and when PostgreSQL or Python is sufficient.

Recommended resources

Apache Spark | PySpark | DuckDB | Apache Iceberg

Exit criterion

You can explain how a Spark job executes, investigate a slow join, and make evidence-based performance improvements.

PHASE 09

Streaming data and event-driven pipelines

Month 9 | Estimated effort: 35–45 hours | Cloud and scale

Batch answers questions about data collected over an interval. Streaming processes events continuously or in small increments, often because freshness matters.

What you will learn

Module 9.1 — Streaming fundamentals

Consider a mobile-money platform: fraud detection or operational monitoring may need events processed soon after they occur, not at the end of the day.

- Events, event time and processing time
- Producers, consumers and brokers; topics and partitions
- Consumer groups and offsets; ordering guarantees
- At-most-once, at-least-once and exactly-once semantics; idempotent consumers
- Event-time windows, late-arriving events, watermarks and stateful processing
- Backpressure and consumer lag
- Schema registries and schema evolution
- Replay and dead-letter queues

Module 9.2 — Apache Kafka

- Brokers and clusters; replication and fault tolerance
- Producers and consumer acknowledgements
- Consumer groups and rebalancing
- Retention policies and offset management
- Key-based partitioning
- Connectors and CDC integration
- Security and monitoring

Do not over-trust delivery guarantees

Kafka does not automatically guarantee every business transaction is processed exactly once. End-to-end guarantees depend on source behavior, broker configuration, consumer logic and the destination's transaction semantics.

Tools, platforms and tech stack

Task	Recommended tool	Free alternative / cost
Event streaming	Apache Kafka	Free and open source
Kafka development	Kafka CLI tools and Python clients	Free
Python client	`confluent-kafka` or `kafka-python`	Free libraries; some vendor services are paid
Stream processing	Spark Structured Streaming	Free

Task	Recommended tool	Free alternative / cost
Event generation	Python	Free
Schema validation	JSON Schema or Avro tooling	Free core tools
Local environment	Docker Compose	Free core tooling
Message inspection	Kafka CLI tools	Free

Free approach: run a single Kafka broker locally in Docker and avoid paid managed Kafka clusters until you have a specific reason. A local setup is excellent for learning, but it is not equivalent to operating a secure, highly available production cluster.

Hands-on project

Project 8 — Real-time event pipeline

Simulate a retail transaction stream:

```
Transaction generator
        |
        v
    Kafka topic
        |
        v
  Stream consumer
        |
        +----> Raw event storage
        |
        +----> Validated transaction store
        |
        +----> Near-real-time aggregates
```

Detect malformed events, preserve event identifiers, handle duplicate deliveries, monitor consumer lag and recover after a consumer restart. Aggregates go to PostgreSQL or Parquet; invalid records go to a dead-letter destination. Build version one with a Python Kafka client, then extend with Spark Structured Streaming if you need distributed processing.

Recommended resources

Apache Kafka | Confluent Python client | Spark Structured Streaming

Exit criterion

You can build and debug a basic event pipeline and explain its delivery, ordering and recovery guarantees.

PHASE 10

Data quality, governance, security and observability

Month 10 | Estimated effort: 35–45 hours | Professional readiness

This phase separates a convincing demonstration from a trustworthy data platform. A dashboard can look perfect while its underlying data is wrong; you must detect that before decisions are made from it.

What you will learn

Module 10.1 — Data quality engineering

Dimension	Question to test
Completeness	Are required values present?
Uniqueness	Are business keys duplicated?
Validity	Do values satisfy defined rules?
Consistency	Do related datasets agree?
Referential integrity	Do foreign keys point to valid records?
Freshness	Is the data recent enough?
Accuracy	Does the data reflect the underlying reality?
Reconciliation	Do totals agree with trusted source records?

Use SQL tests, Python tests, dbt tests and a data-quality framework when the added functionality justifies it.

Module 10.2 — Observability

Monitor pipeline success and failure rates, duration and resource consumption, source-to-destination record counts, data freshness, volume anomalies, schema changes, retry rates, streaming lag, warehouse query performance, and cost and storage growth.

Learn structured logging, metrics, alerts, run identifiers and incident reports. Prometheus and Grafana help you understand infrastructure and service behavior; data-quality tools give complementary visibility into the datasets themselves.

Module 10.3 — Security and governance

- Least-privilege and role-based access control
- Encryption in transit and at rest; secrets management
- Personally identifiable information (PII); data minimization and retention
- Audit logging and data classification
- Data lineage and ownership
- Backup and disaster recovery
- Data contracts and schema compatibility

Kenyan context

For Kenyan projects, understand the principles of Kenya's Data Protection Act, 2019 and the responsibilities that can apply to organizations handling personal data. Consult the Office of the Data Protection Commissioner for authoritative guidance.

Tools, platforms and tech stack

Task	Recommended tool	Free alternative / cost
Unit and integration tests	pytest	Free
SQL transformation tests	dbt tests	Free open-source workflow
Data validation	Great Expectations	Open-source components; check current product tiers
Alternative validation	Pandera	Free and open source
Metrics	Prometheus	Free and open source
Monitoring dashboards	Grafana	Free self-hosted edition
Logs	Python logging / structured JSON logs	Free
Error tracking	Sentry	Free tier subject to current limits; logs are an alternative
Data lineage	dbt documentation and dependency graph	Free
Secrets	Cloud secret manager	Environment variables for local learning
Governance documentation	Markdown, YAML, data dictionaries	Free

You do not need a monitoring stack for every small script; structured logs and well-written tests are enough to start. **Cost-saving rule:** prefer self-hosted open-source tools while learning, since managed observability platforms become expensive at high log or metric volumes.

Hands-on project

Project 9 — Engineer a quality-control layer

Extend an existing pipeline with:

- A documented set of quality rules
- Automated tests at ingestion and transformation stages
- Source-to-target reconciliation
- Alerts when critical checks fail
- A quarantine area for invalid records
- A data freshness report
- An incident runbook and a recovery and backfill procedure

Completion test: introduce a deliberate source-data defect. Show how the system detects it, keeps invalid data out of the final analytical model, reports the issue and recovers after correction.

Recommended resources

Great Expectations | Pandera | Prometheus | Grafana | Kenya ODPC

Exit criterion

You can detect, contain, report and recover from a bad-data incident before it reaches business consumers.

PHASE 11

Production engineering, CI/CD and advanced design

Month 11 | Estimated effort: 40–50 hours | Professional readiness

You should now have several working pipelines. Learn how teams safely maintain and deploy them.

What you will learn

Module 11.1 — Software engineering for data systems

- Modular architecture and separation of concerns
- Unit, integration and end-to-end tests; deterministic test data
- Configuration for development, testing and production
- Dependency pinning and semantic versioning
- Code review and branching workflows
- Documentation, runbooks, refactoring and technical debt
- Backward-compatible schema changes

Module 11.2 — CI/CD

Learn GitHub Actions or an equivalent platform and build a workflow that:

1. Runs Python linting and type checks
2. Executes automated tests
3. Validates SQL or dbt models
4. Builds a container image
5. Checks dependencies for known vulnerabilities
6. Publishes an image or deployment artifact
7. Deploys to a safe test environment
8. Promotes approved changes to production

Keep deployment credentials out of source control; use short-lived credentials or protected environment secrets where supported.

Module 11.3 — Reliability and system design

- Service-level indicators, objectives and agreements
- Availability and recovery objectives; fault isolation; graceful degradation
- Disaster recovery and restore testing
- Data retention and reprocessing
- Capacity planning and cost-performance trade-offs
- Rate limiting and resource quotas
- Multi-environment architecture; safe schema and pipeline migrations

Module 11.4 — Data architecture and trade-offs

Practise answering questions such as:

- When should you use PostgreSQL instead of a warehouse?

- When is a scheduled batch pipeline sufficient, and when is streaming worth its complexity?
- When should data be stored as Parquet, when does Spark make sense, and when is a managed cloud service worth it?
- How do you handle late-arriving data or recover a corrupted historical partition?
- How do you prevent a rerun from double-counting revenue?
- How do you balance freshness, cost, reliability and maintainability?

There is rarely one universally correct architecture. The strongest engineers justify choices using requirements, constraints and measurable trade-offs.

Tools, platforms and tech stack

Task	Recommended tool	Free alternative / cost
Source control	Git + GitHub	Free plans available
CI/CD	GitHub Actions	Free allowance; limits depend on repository and runner type
Containerization	Docker	Free core tooling
Infrastructure as Code	Terraform	OpenTofu
Automated testing	pytest + dbt tests	Free
Python linting	Ruff	Free
Dependency security	pip-audit	Free
Container scanning	Trivy (config via YAML / env vars)	Free open-source scanner
Documentation	Markdown + diagrams.net	Free

Do not learn Kubernetes yet

Not until you can comfortably build, containerize, test and operate a single pipeline. Kubernetes is valuable in certain environments, but it is not a prerequisite for every junior Data Engineering role. You can demonstrate a complete CI workflow with free tools without deploying to the cloud.

Hands-on project

Project 10 — Productionize an existing pipeline

Take one earlier project and add:

- Automated testing in CI and containerized execution
- Configuration-driven deployments
- Structured logs and meaningful error messages
- Monitoring and failure alerts; secure credential handling
- A backup and recovery procedure
- A runbook, architecture diagram and documented cost estimate
- A successful recovery demonstration

Recommended resources

GitHub Actions | OpenTofu | pip-audit | Trivy | kind

Exit criterion

Another engineer can clone your repository, understand it, run the tests, deploy it safely and diagnose a simulated failure without undocumented instructions from you.

PHASE 12

The industry-grade capstone

Month 12 | Estimated effort: 50–70 hours | Professional readiness

Combine everything into one coherent data platform: locally relevant, technically substantial and easy for a recruiter to understand.

What you will learn

Capstone: Kenya Retail Intelligence Data Platform

Business scenario: a retail business operates multiple branches and struggles with inconsistent sales reports, delayed inventory updates, stockouts and poor visibility into product performance. Build a platform that collects, processes, validates, stores and serves trustworthy data for decision-making.

Layer	Components
Data sources	Public retail datasets, API records and simulated transactions
Ingestion layer	Python, API clients, incremental loads and Kafka events
Storage layer	PostgreSQL and object storage using Parquet
Transformation and orchestration	dbt, Airflow and PySpark where justified
Quality and operations	Automated tests, observability, access controls and CI/CD
Analytics and consumers	Power BI, SQL analytics, reporting APIs and forecasting-ready datasets

A learning target, not a deployment requirement

Start with a dependable batch pipeline. Add streaming, Spark and cloud services only when you can explain what each addition contributes.

Capstone acceptance checklist

Done	Area	What must be demonstrated
☐	Data ingestion	At least two data sources or source types, incremental extraction and raw-data preservation
☐	Database and warehouse	Well-designed schemas, explicit table grain, fact and dimension models and historical tracking
☐	Pipeline orchestration	Scheduled execution, dependency management, retries, backfills and safe reruns
☐	Data transformation	Maintainable Python and SQL; dbt models with documentation and tests
☐	Data quality	Reconciliation, duplicate detection, schema validation, freshness checks and error quarantine
☐	Cloud deployment	A working deployment or reproducible cloud configuration with cost controls

Done	Area	What must be demonstrated
☐	Testing and CI/CD	Automated tests, version control, repeatable builds and safe deployment practices
☐	Observability	Execution logs, metrics, failure reporting and a recovery runbook
☐	Business value	Documented metrics answering real questions about sales, inventory and branch performance
☐	Professional documentation	Architecture diagram, setup instructions, data dictionary, design decisions and a demonstration

Suggested data sources

- Rossmann Store Sales (Kaggle): retail sales and forecasting exercises
- World Bank Open Data: economic and development data
- Kenya National Bureau of Statistics: official statistics and publications
- Our World in Data: datasets and research

Check each dataset's licence before redistributing it. For realistic inventory movements, branch identifiers and supplier records, generate synthetic data to complement a public sales dataset and label it clearly as simulated.

Tools, platforms and tech stack — Recommended capstone stack

Layer	Tool	Cost
Data sources	Public datasets + Python-generated events	Free
Batch ingestion	Python + Requests	Free
Streaming ingestion	Apache Kafka	Free locally
Raw storage	MinIO locally; Amazon S3 in the cloud	Local free; cloud storage may cost
Relational storage	PostgreSQL	Free locally
Data transformation	SQL + dbt Core	Free
Distributed processing	PySpark	Free locally
Workflow orchestration	Apache Airflow	Free locally
Containers	Docker Compose	Free core tooling
Infrastructure	Terraform / OpenTofu	Free
Data quality	pytest + dbt tests + Pandera	Free
Observability	Structured logs + Prometheus + Grafana	Free when self-hosted
CI/CD	GitHub Actions	Free allowance available
Analytics	Power BI Desktop or Metabase	Free starting options
Documentation	GitHub README + Markdown + diagrams.net	Free

Use one coherent stack rather than combining technologies at random. Your batch pipeline should work before you add Kafka, your SQL should be sound before you add Spark, and your local pipeline should be tested before you deploy to the cloud.

Suggested repository structure

```
kenya-retail-data-platform/
├── ingestion/
│   ├── extract_api.py
│   ├── load_postgres.py
│   └── kafka_producer.py
├── transformations/
│   ├── python/
│   └── dbt_project/
├── streaming/
│   └── consumer.py
├── orchestration/
│   └── dags/
├── storage/
│   ├── schemas/
│   └── sample_data/
├── tests/
│   ├── unit/
│   └── integration/
├── infrastructure/
│   └── terraform/
├── monitoring/
├── docker-compose.yml
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md
```

Keep real credentials out of `.env.example` and every committed file. Include placeholder variable names and document how a developer creates their own local configuration.

> **Exit criterion**
>
> A recruiter can understand the platform in minutes, and another engineer can run it, test it, break it and recover it.

Your weekly study system

Assuming about 15 hours per week, use this pattern. Across 12 months that is roughly 780 study hours: a substantial foundation, though some topics will take longer than planned.

Hours	Activity	What it looks like
4	Learn	Study documentation and lessons; take notes in your own words.
5	Practice	Solve SQL problems, write scripts, reproduce examples and test edge cases.
4	Build	Implement your current project and integrate what you have learned.
2	Review	Debug, revise notes, document progress and explain your design decisions.

A typical learning week

Day	Focus
Monday	Study one new concept and reproduce a small example
Tuesday	Complete guided exercises
Wednesday	Solve a problem without following a tutorial
Thursday	Add the concept to your project
Friday	Write tests and troubleshoot failures
Saturday	Extend the project and document your work
Sunday	Review, rest or catch up

Every week should produce tangible evidence of progress: code, tests, a diagram, a written explanation or a working feature.

Readiness self-check

At the end of each phase, rate each skill honestly: **0** = cannot do it yet, **1** = can do it with guidance, **2** = can do it independently, **3** = can do it independently and explain the trade-offs.

Skill	0	1	2	3
1. Write and debug Python pipeline code independently	☐	☐	☐	☐
2. Write complex SQL and explain query execution plans	☐	☐	☐	☐
3. Design databases and analytical models with correct grain	☐	☐	☐	☐
4. Build incremental, idempotent, recoverable pipelines	☐	☐	☐	☐
5. Orchestrate workflows and troubleshoot failed tasks	☐	☐	☐	☐
6. Deploy a secure, cost-aware cloud data pipeline	☐	☐	☐	☐
7. Explain and optimize distributed processing where needed	☐	☐	☐	☐
8. Test, monitor, document and recover a data platform	☐	☐	☐	☐

Self-assessment score: _____ / 24

Self-reported, not a certification. A high score is meaningful only if you can demonstrate the skills in a working project, explain your decisions and troubleshoot problems you have not seen before.

Build a portfolio employers can evaluate

Three substantial, well-engineered projects are worth more than ten superficial repositories. Build these three pillars:

Portfolio pillar	Stack	What to demonstrate
1 — Reliable batch ETL	Python, SQL, PostgreSQL, testing, incremental loading, Airflow	Data extraction, deduplication, recovery, reconciliation and maintainable code
2 — Cloud analytics platform	AWS, object storage, Parquet, dbt, Terraform, BI	Secure cloud storage, reproducible infrastructure, curated datasets and cost awareness
3 — Event-driven data pipeline	Kafka, Python, streaming, data quality, observability	Event processing, duplicate handling, recovery, monitoring and clear delivery guarantees

Every repository should contain

- A clear problem statement and business use case
- An architecture diagram and a reproducible setup guide
- Sample data or instructions for obtaining it
- Meaningful tests and sample outputs
- Data-quality rules and known limitations
- Design decisions and alternatives considered
- Performance measurements where relevant
- A short demonstration and screenshots where useful

Never commit secrets

Never commit passwords, API keys, database connection secrets or private data to a public repository.

Affordable learning strategy

You can learn a substantial portion of Data Engineering without boot camps or expensive certifications. Start locally with Python, PostgreSQL, DBeaver, Git, Docker, dbt and Airflow, using public datasets, synthetic data, free documentation and small workloads. For cloud experience, deploy only when a project requires it: check pricing first, set budget alerts and tear down resources after each exercise.

Where to spend, and where not to

Use free local tools by default

Python, PostgreSQL, DBeaver Community, Git, dbt Core, Airflow, Spark, Kafka and Terraform/OpenTofu cover a large proportion of the curriculum without subscription costs.

Use free cloud credits strategically

Use AWS or Azure credits for a few targeted deployment exercises. Do not keep databases, virtual machines, warehouses or clusters running when idle. Check current eligibility and service limits first.

Pay for training only when it solves a specific problem

Start with official documentation and free courses. Consider a paid course only if it offers structured instruction, meaningful exercises, feedback or access you cannot obtain elsewhere.

Delay certifications until your portfolio is credible

A certification can support your job search but should not replace practical projects. Decide on spending once you know which Kenyan or remote employers and roles you are targeting.

Free software is not free infrastructure: self-hosting costs electricity, storage, memory, maintenance and your time, while managed platforms charge for convenience, scale and operations. Learn the difference between the cost of a tool and the cost of operating a system.

A sensible sequence

1. Build strong fundamentals using free resources
2. Complete the batch ETL and warehouse projects
3. Add orchestration and testing
4. Deploy one small cloud pipeline
5. Apply for internships, junior roles and practical opportunities while continuing to improve
6. Consider a targeted cloud certification when it supports your job-search strategy

The master tool list

Area	Primary tool	What you need to demonstrate
Programming	Python	Reliable, testable pipeline code
Querying	SQL	Complex transformations and optimization
Database	PostgreSQL	Database design and administration fundamentals
Version control	Git/GitHub	Collaboration and reproducibility
Data processing	pandas, PyArrow	Structured data processing and file formats
Transformation	dbt	Tested analytical models
Orchestration	Airflow	Scheduling and recovery
Containers	Docker	Reproducible environments
Cloud	AWS	Secure, cost-aware deployments
Distributed processing	PySpark	Scale-oriented processing and optimization
Streaming	Kafka	Event ingestion and delivery semantics

Area	Primary tool	What you need to demonstrate
Infrastructure as code	Terraform	Reproducible infrastructure
Quality	SQL/dbt tests; Great Expectations when useful	Detecting and preventing bad data
CI/CD	GitHub Actions	Automated validation and deployment
Analytics	Power BI	Demonstrating the business value of your data

Free-first installation checklist

Install progressively on a Windows 11 development machine. Airflow, Spark and Kafka can consume substantial memory together, so avoid running them all at once on a resource-constrained laptop.

Done	Step	Install
☐	1. Development environment	VS Code, Git, Ubuntu on WSL2, Python and a virtual environment
☐	2. Database environment	PostgreSQL, DBeaver Community and psql
☐	3. Python data tools	pandas, PyArrow, Requests, SQLAlchemy, Psycopg, pytest and Ruff
☐	4. Data modeling	dbt Core, a dbt PostgreSQL adapter and a small retail dataset
☐	5. Containerization	Docker Desktop if eligible for free use (or another Docker Engine environment), then Docker Compose
☐	6. Orchestration	Apache Airflow running locally through Docker Compose
☐	7. Cloud fundamentals	AWS account if eligible, IAM, S3, billing controls and a small test workload
☐	8. Advanced processing	PySpark, Kafka and MinIO, introduced one at a time
☐	9. Production practices	GitHub Actions, Terraform or OpenTofu, quality checks and monitoring

Priorities and the final tool-selection rule

Before adopting any new technology, ask

1. What problem does this tool solve?
2. Can I demonstrate that problem using my existing stack?
3. What additional complexity or cost will this tool introduce?

For example: use PostgreSQL before a cloud warehouse for foundational SQL, Python before Airflow for pipeline logic, DuckDB before Spark for many single-machine analytical tasks, and a local Kafka broker before paying for a managed streaming platform. This builds engineering judgment rather than familiarity with software names.

If you were starting from zero, the order is

Step	Focus	Why
1	Python + SQL + Git + Linux	A reliable foundation before adding frameworks
2	PostgreSQL + data modeling + ETL	Extract, transform, store and validate data correctly
3	dbt + Airflow + Docker	Turn individual scripts into a maintainable, automated pipeline
4	AWS + object storage + Terraform	Deploy and operate the platform securely
5	Spark + Kafka	Add distributed and streaming processing once you know why you need them
6	Testing + observability + production practices	Make systems trustworthy, recoverable and maintainable
7	Capstone + portfolio + job applications	Demonstrate what you can build and explain

One final professional truth

An outstanding Data Engineer is not the person who uses the most sophisticated technology. It is the person who solves the business problem with the simplest reliable architecture, keeps data trustworthy, recovers gracefully from failures, controls costs and explains the trade-offs clearly.

Follow this curriculum consistently, build the projects rather than merely watching tutorials, and practise debugging independently. Your goal is not just to become employable; it is to become the engineer people trust with the data systems their businesses depend on.